

TENXO INTERNAL ARCHITECTURE

System Design

Tenxo is a decentralized GPU compute marketplace with four main planes:

- Frontend: Next.js app for providers, consumers, marketplace, billing, and account workflows.
- Matchmaker API: Go HTTP service that owns auth, job state, node state, billing integration, storage presign, and WebSocket job updates.
- Orchestration bus: NATS JetStream work queue for job dispatch plus NATS subjects for node heartbeat and job result events.
- Workers and clients: Rust edge agent advertises GPUs and executes jobs; Python CLI/client packages and the web app submit encrypted workloads.

PostgreSQL is the source of truth for nodes, jobs, API keys, usage records, billing customers, and payment transaction history. Object storage is Cloudflare R2 in production, with local encrypted upload storage for development.

Services

- 'frontend': Next.js 14 UI. Uses Supabase auth, matchmaker REST APIs, and '/ws' for live job lifecycle updates.
- 'backend': Go matchmaker. Validates Supabase JWTs or Tenxo API keys, lists inventory, creates jobs, signs storage URLs, stores usage, and proxies Razorpay billing operations.
- 'edge_agent': Rust worker. Publishes heartbeats, reports GPU metadata and TEE status, consumes work, uploads encrypted results.
- 'tenxo': Python package and CLI. Packs/encrypts workloads, calls presign/job APIs, and downloads encrypted results.
- 'nats': JetStream broker. 'jobs' stream uses work-queue retention for worker consumption; 'jobs.results' reports completion.
- 'postgres': Persistent metadata and billing state.
- 'r2/object storage': Encrypted job payload and result blobs.

APIs

- 'GET /health': service liveness.
- 'GET /install.sh': provider install script.
- 'GET /nodes': authenticated marketplace inventory.
- 'GET /my-nodes': authenticated provider-owned nodes.
- 'GET /metrics': authenticated dashboard metrics and GPU SKU inventory.

- 'POST /agent/heartbeat': authenticated provider heartbeat.
- 'POST /presign': authenticated encrypted upload/result URLs.
- 'GET /jobs': authenticated owner-scoped job list.
- 'POST /jobs': authenticated encrypted job submission.
- 'GET /jobs/{id}/status': authenticated owner-scoped job detail.
- 'PUT|POST /storage/upload/{id}': authenticated local dev encrypted upload.
- 'PUT|POST /storage/result-upload/{id}': authenticated local dev encrypted result upload.
- 'GET /storage/result/{id}': authenticated owner-scoped result download.
- 'GET /ws': WebSocket job update channel, authenticated by first message.
- 'GET|POST /billing/*': authenticated Razorpay customer, token, usage, charge, and transaction APIs.
- 'POST /billing/webhook': Razorpay webhook with HMAC signature verification.
- 'GET|POST /api/keys': API key listing and creation.
- 'DELETE|PATCH /api/keys/{hash}': API key revocation and rename.
- '/signal/*': zero-knowledge key exchange signaling routes.

Node Orchestration

1. Provider installs and runs the edge agent with a Tenxo API key.
2. Agent POSTs '/agent/heartbeat' or publishes 'heartbeats.{node_id}' with status, GPU model, VRAM, owner, and attestation state.
3. Matchmaker persists node state and exposes fresh nodes with a 90-second liveness window.
4. Consumer uploads encrypted workload through '/presign', then submits '/jobs'.
5. Matchmaker stores owner-scoped job metadata and publishes a 'jobs' JetStream message.
6. Worker claims the job, executes it, uploads encrypted results, and publishes 'jobs.results'.
7. Matchmaker updates job state and broadcasts the event to the consumer over '/ws'.

Payment Flow

1. Consumer opens billing and the frontend calls '/billing/customer'.
2. Backend creates or reuses a Razorpay customer bound to the authenticated user.
3. '/billing/setup-intent' creates a small verification order.
4. Razorpay Checkout returns order, payment, and signature values.
5. '/billing/verify-payment' validates the HMAC signature and stores the returned token.
6. Usage is started/stopped by trusted job lifecycle events through '/billing/track-usage'.
7. '/billing/charge' or auto-charge creates a Razorpay order and recurring payment with the saved token.
8. '/billing/webhook' records captured payments and updates paid totals.
9. '/billing/transactions' returns usage and payment history for the dashboard.

GPU Allocation Flow

Current allocation is queue-oriented. Available idle nodes are surfaced through '/nodes' and '/metrics'; jobs enter the 'jobs' JetStream work queue with optional requested GPU metadata. Workers decide eligibility based on local GPU inventory and job requirements. Production scaling should add explicit reservation records with node locks, requested SKU, memory floor, lease TTL, and idempotent release on completion or timeout.

Deployment Topology

Production topology:

- CDN or platform routing terminates TLS for the Next.js frontend.
- Matchmaker API runs behind HTTPS with 'ALLOWED_ORIGINS' pinned to frontend origins.
- PostgreSQL runs in a private network with backups and restricted credentials.
- NATS JetStream runs with file storage, auth, persistence, and private networking.
- R2/S3-compatible storage holds encrypted payloads and results.
- Edge agents connect outbound only to HTTPS/NATS endpoints.
- Razorpay webhook endpoint is public but HMAC verified.

Security controls:

- Supabase JWT or hashed 'txn_' API key auth on protected routes.
- Owner-scoped job, node, billing, and storage reads.
- API keys are stored as SHA-256 hashes.
- Webhooks and payment verification use HMAC checks.
- Local file storage sanitizes IDs and confines paths to the upload root.
- Browser CORS and WebSocket origins must be restricted in production.